

TASKFLOW

Documentación Técnica del Sistema

Arquitectura, Modelos de Datos, Base de Datos y Migraciones

Presentado por:

Oscar Camargo

Kevin Zapa

Juan Fayad

Universidad Francisco José de Caldas

Entregado a:

Ing. Jonathan Roberto Torres Castillo, PhD

Programación

Avanzada

Fecha de entrega: 1 / Junio / 2026

Bogotá D.C., Colombia

Resumen

Este documento presenta la documentación técnica del sistema TaskFlow, una aplicación de gestión de proyectos y seguimiento de tareas desarrollada con Flask y SQLAlchemy. Se describen su arquitectura general, los modelos de datos, la capa de persistencia, el sistema de migraciones y los principales componentes que conforman la solución.

Motivos del proyecto

En la creación de proyecto se tuvo en cuenta los requisitos que son basados en las temáticas de esta materia, sobre todo fijandonos como temas de interés las bases de datos (persistencia), concurrencia, MVC con plantillas html.

Teniendo en cuenta esos requisitos y luego pensando en nuestras necesidades surge una idea simple y útil, un programa de gestión de proyectos, siendo así nuestro público objetivo académicos y personas con agendas digitales ajustadas.

Vemos en nuestra aplicación una solución que no solo almacena tareas sino permite visualizar y priorizar cada proyecto.

Público objetivo

Task flow esta dirigida a:

- Equipos de trabajo medianos o pequeños.
- Grupos de desarrollo pequeños.
- Organizaciones que necesitan seguimiento de tareas.
- Administradores que requieren supervisar tareas.

¿Por qué es una buena solución?

Porque el sistema es capaz de centralizar la gestión del trabajo con herramientas de tareas cortas para afrontar grandes problemas de a poco facilitando el trabajo cooperativo, el menú no permite que olvides las tareas y te deja un rastro de deberes realizados y por realizar, además que el programa gracias a su modularidad puede escalar y crecer.

Descripción General de TaskFlow

Archivos fuente relevantes

TaskFlow es una aplicación de gestión de proyectos y seguimiento de tareas diseñada con una arquitectura en capas utilizando el framework Flask. Proporciona un sistema de doble interfaz compuesto por un panel web renderizado en el servidor (SSR) y una API RESTful en formato JSON. El sistema está construido para gestionar usuarios, proyectos y tareas, con capacidades integradas de auditoría y notificaciones.

Propósito y alcance

El objetivo principal de TaskFlow es proporcionar una plataforma robusta para la gestión colaborativa de tareas. Incluye una interfaz temática llamada **"Cyberpunk Edition"** para los usuarios finales y una API estructurada para acceso programático. El sistema enfatiza la integridad de los datos mediante una capa de acceso basada en repositorios y servicios automáticos en segundo plano para monitoreo del sistema y alertas a los usuarios.

Stack tecnológico

TaskFlow utiliza una pila tecnológica moderna basada en Python:

- **Framework:** Flask
 - `app/__init__.py`
- **ORM de base de datos:** SQLAlchemy
 - `app/models/__init__.py`
 - mediante Flask-SQLAlchemy (`requirements.txt`)
- **API:** Flask-RESTful
 - `requirements.txt`
- **Migraciones:** Alembic mediante Flask-Migrate
 - `requirements.txt`

- **Motor de plantillas:** Jinja2
 - `requirements.txt`
 - **Base de datos:** SQLite
 - `app/__init__.py`
-

Arquitectura del Sistema

La aplicación sigue el patrón **Application Factory**, donde la función `create_app()` inicializa los componentes principales, incluyendo la base de datos, las rutas web y los endpoints de la API.

Interacción de componentes de alto nivel

El siguiente diagrama ilustra cómo interactúan los principales subsistemas dentro del contexto de la aplicación Flask.

Interacción de subsistemas de TaskFlow

Fuentes:

- `app/__init__.py`
 - `app/config.py`
-

Subsistemas principales

1. Capa de acceso a datos

TaskFlow utiliza el patrón **Repository** para desacoplar las capas web y API de los modelos SQLAlchemy subyacentes. Esto garantiza que la lógica de negocio relacionada con la recuperación y persistencia de datos esté centralizada.

Modelos

Definen el esquema para:

- User
- Project
- Task
- Notification

- LogEntry

Repositorios

Proporcionan interfaces limpias para operaciones CRUD, tales como:

- UserRepository
 - TaskRepository
-

2. Interfaces Web y API

El sistema expone su funcionalidad mediante dos mecanismos principales:

Rutas Web

Gestionan envíos de formularios tradicionales y renderizan plantillas Jinja2 para el panel de control.

- `app/__init__.py`

API REST

Proporciona endpoints JSON para integración con herramientas externas.

- `app/__init__.py`

Ejemplos de endpoints:

- `GET /api/projects`
 - `POST /api/tasks`
-

3. Servicios en segundo plano

TaskFlow está diseñado para ejecutar hilos daemon que manejan tareas asíncronas:

Servicio de Notificaciones

- Supervisa las fechas límite de las tareas.
- Genera alertas para elementos vencidos.

Servicio de Logs

- Guarda periódicamente los registros de auditoría del sistema en almacenamiento persistente.

Nota: Estos servicios se inicializan dentro del contexto de la Application Factory.

- `app/__init__.py`
-

Relación entre código y sistema

El siguiente diagrama relaciona entidades específicas del código con sus funciones dentro del sistema.

Diagrama de mapeo de entidades

Fuentes:

- `app/__init__.py`
 - `run.py`
-

Secciones de documentación detallada

Para información más específica sobre áreas concretas del código, consultar las siguientes páginas:

Primeros pasos

Guía paso a paso para:

- Configurar el entorno.
- Instalar dependencias desde `requirements.txt`.
- Inicializar la base de datos SQLite `taskflow.db`.

Estructura del proyecto

Desglose de la estructura de directorios y explicación de:

- `app/`
- `migrations/`

- `data/`

Fuentes:

- `app/__init__.py`
 - `README.md`
 - `app/config.py`
 - `run.py`
-

Arquitectura Central (Core Architecture)

Archivos fuente relevantes

Esta sección proporciona una visión general de la estructura interna de TaskFlow.

TaskFlow está construido utilizando una **arquitectura en capas** que separa:

- Persistencia de datos.
- Lógica de negocio.
- Presentación.

El sistema sigue el patrón **Application Factory** para gestionar su ciclo de vida y configuración.

Ciclo de vida de la aplicación

TaskFlow utiliza el patrón **Flask Application Factory** mediante la función `create_app()`.

Este enfoque garantiza que la instancia de la aplicación se cree con:

- La configuración correcta.
- La URI de la base de datos.
- Las claves secretas.

Todo esto antes de registrar servicios o rutas.

Secuencia de inicialización

La inicialización sigue un orden estricto para garantizar que todas las dependencias estén disponibles:

1. Configuración del entorno

Asegura que exista la ruta de instancia para la base de datos SQLite.

- `app/__init__.py`

2. Configuración

Define:

- `SQLALCHEMY_DATABASE_URI`
- `SECRET_KEY`
- `app/__init__.py`

3. Vinculación de la base de datos

Conecta la instancia de SQLAlchemy con la aplicación Flask.

- `app/__init__.py`

4. Registro de rutas

Carga las rutas web basadas en Jinja2.

- `app/__init__.py`

5. Registro de la API

Carga los endpoints RESTful.

- `app/__init__.py`

6. Creación del esquema

Ejecuta:

`db.create_all()`

dentro del contexto de la aplicación para garantizar que el archivo SQLite esté listo.

- `app/__init__.py`

Para más detalles sobre variables de configuración y el proceso de creación de la aplicación, consultar la documentación de **Application Factory y Configuración**.

Arquitectura en capas

TaskFlow está organizado en capas claramente diferenciadas para mantener la separación de responsabilidades.

Esto permite ofrecer simultáneamente:

- Una interfaz web tradicional.
- Una API JSON programática.

Ambas utilizando los mismos modelos de datos subyacentes.

Diagrama de capas del sistema

El siguiente diagrama muestra cómo una solicitud fluye a través de las distintas entidades de código.

Flujo de solicitud: Usuario → Base de datos

Fuentes:

- `app/__init__.py`
 - `app/models/__init__.py`
-

Descripción de las capas

Capa	Función	Entidades principales
Modelos	Define el esquema de datos y relaciones.	User, Project, Task, Notification, LogEntry
Repositorios	Gestiona acceso a datos y abstrae consultas SQLAlchemy.	UserRepository, ProjectRepository, TaskRepository

Rutas Web	Gestiona solicitudes HTTP y renderiza plantillas Jinja2.	<code>init_routes()</code> , <code>auth.py</code> , <code>projects.py</code>
API REST	Proporciona endpoints JSON para acceso programático.	<code>register_api()</code> , <code>ProjectListAPI</code> , <code>TaskListAPI</code>
Servicios en segundo plano	Supervisa tareas y registra actividad del sistema en hilos separados.	<code>NotificationService</code> , <code>LogService</code>

Entidades de datos y persistencia

El núcleo del sistema gira alrededor de cinco modelos SQLAlchemy que definen la lógica del dominio.

Estos modelos están centralizados en el paquete:

`app.models`

Mapeo entre modelos y tablas de base de datos

Fuentes:

- `app/models/__init__.py`

Para un análisis detallado de:

- Esquemas.
- Restricciones de claves foráneas.
- Métodos de serialización `to_dict()`.

Consultar la sección **Modelos de Datos**.

Patrón Repository

TaskFlow utiliza el patrón **Repository** para desacoplar las capas Web y API de la lógica de `db.session`.

En lugar de llamar directamente a:

```
db.session.add()
```

desde una ruta, el código invoca métodos de una clase repositorio.

Ventajas

Desacoplamiento

Las rutas no necesitan conocer los detalles de los filtros y consultas de SQLAlchemy.

Reutilización

La misma lógica para operaciones como:

"obtener tareas vencidas"

puede compartirse entre:

- La interfaz de usuario.
- Los servicios en segundo plano.

Seguridad

Centraliza funcionalidades críticas como:

- Hashing de contraseñas.
- Verificación de credenciales.

Para detalles sobre los métodos CRUD y consultas especializadas, consultar la sección **Capa de Repositorios**.

Modelos de Datos (Data Models)

Archivos fuente relevantes

La capa de datos de TaskFlow está construida utilizando **SQLAlchemy**, proporcionando un sistema de Mapeo Objeto-Relacional (ORM) que abstrae la base de datos SQLite subyacente.

El sistema define cinco entidades principales que gestionan:

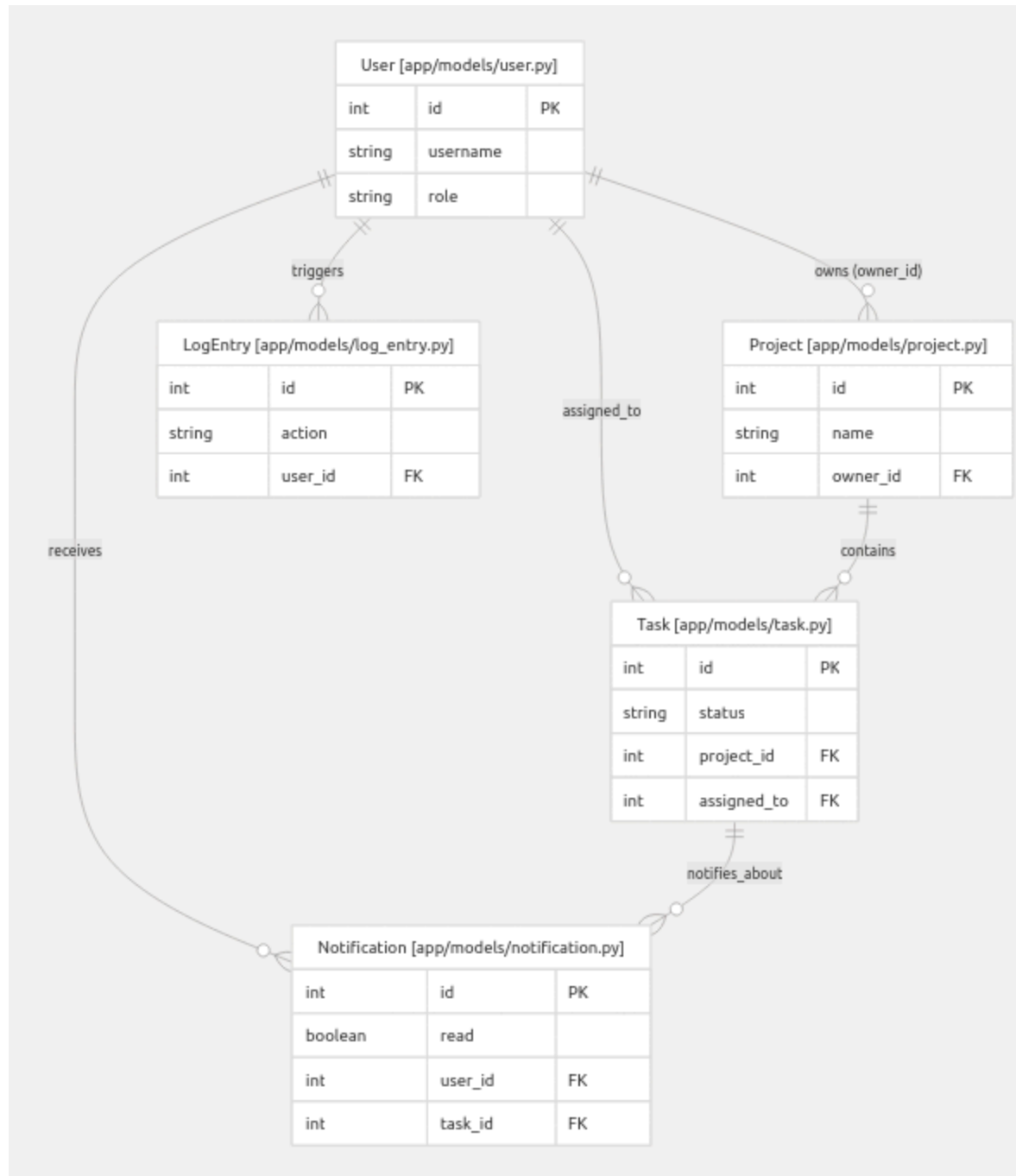
- Identidad de usuarios.

- Organización de proyectos.
- Ciclo de vida de las tareas.
- Alertas automáticas.
- Auditoría del sistema.

Visión general de relaciones entre entidades

El siguiente diagrama ilustra las relaciones entre los cinco modelos, destacando las restricciones de claves foráneas (Foreign Keys) y el mapeo dentro del espacio de entidades del código.

Diagrama Entidad-Relación (ER) de TaskFlow



Categorías principales de modelos

Modelos User y Project

User

El modelo **User** administra la autenticación y autorización de usuarios, soportando roles como:

- `admin`
- `member`

`app/models/user.py`

Project

El modelo **Project** funciona como contenedor para el trabajo colaborativo.

Cada proyecto debe tener obligatoriamente un campo:

`owner_id`

que referencia a un usuario (`User`).

`app/models/project.py`

Características clave

Reglas de cascada (Cascade Rules)

Las relaciones están configuradas para eliminar automáticamente registros relacionados:

- Al eliminar un **User**, también se eliminan sus proyectos.
- Al eliminar un **Project**, se eliminan todas sus tareas asociadas.

Esto se logra mediante:

`cascade='all, delete-orphan'`

Referencias:

`app/models/user.py`

`app/models/project.py`

Serialización

Tanto **User** como **Project** implementan un método:

`to_dict()`

para convertir objetos en diccionarios que puedan enviarse como respuestas JSON de la API.

El modelo **Project** incluye además un parámetro opcional:

`include_tasks`

que permite incluir las tareas asociadas durante la serialización.

Referencia:

`app/models/project.py`

Para más detalles consultar:

User and Project Models

Modelos Task, Notification y LogEntry

Task

El modelo **Task** representa la unidad principal de trabajo dentro del sistema.

Permite gestionar aspectos como:

- Estado (`status`)
- Prioridad (`priority`)
- Archivos adjuntos

Ejemplos de estados:

- `pendiente`
- `completada`

Referencias:

`app/models/task.py`

Soporte para archivos

Las tareas pueden almacenar documentos asociados mediante el campo:

`document_filename`

`app/models/task.py`

Notification

El modelo **Notification** almacena alertas generadas para los usuarios.

Generalmente son creadas automáticamente por servicios en segundo plano para informar eventos como:

- Tareas vencidas.
- Recordatorios.
- Eventos importantes del sistema.

Referencia:

`app/models/notification.py`

LogEntry

El modelo **LogEntry** implementa la auditoría del sistema.

Registra acciones realizadas utilizando constantes estandarizadas, por ejemplo:

`ACTION_TASK_CREATED`

`ACTION_USER_LOGIN`

Referencia:

`app/models/log_entry.py`

Funciones de soporte de estos modelos

Alertas automáticas

Las entradas de **Notification** son utilizadas para informar a los usuarios sobre eventos relevantes, especialmente aquellos detectados por servicios automáticos.

Ejemplos:

- Tareas vencidas.
- Fechas límite próximas.
- Cambios importantes en proyectos.

Referencia:

app/models/notification.py

Registro de auditoría

Las entradas de **LogEntry** mantienen un historial global de actividades dentro del sistema.

Esto permite:

- Rastrear acciones de usuarios.
- Registrar eventos del sistema.
- Facilitar auditorías y depuración.

Referencia:

app/models/log_entry.py

Integración del sistema: Mapeo entre código y entidades

El siguiente diagrama conecta los modelos conceptuales de datos con su implementación dentro de las capas de repositorios y servicios.

Flujo de integración de modelos (Model Integration Flow)

Este flujo muestra cómo:

1. Los modelos SQLAlchemy representan los datos.
2. Los repositorios encapsulan el acceso a dichos modelos.
3. Los servicios utilizan los repositorios para ejecutar lógica de negocio.
4. Las capas Web y API consumen los servicios para exponer funcionalidad a los usuarios.

En resumen:

Base de Datos SQLite

↑

Modelos

↑

Repositorios

↑

Servicios

↑

Web Routes / API

↑

Usuario

Esta arquitectura mantiene una clara separación de responsabilidades y facilita el mantenimiento, las pruebas y la reutilización del código.

Modelos de Datos (Data Models)

Archivos fuente relevantes

La capa de datos de TaskFlow está construida utilizando **SQLAlchemy**, proporcionando un sistema de Mapeo Objeto-Relacional (ORM) que abstrae la base de datos SQLite subyacente.

El sistema define cinco entidades principales que gestionan:

- Identidad de usuarios.
- Organización de proyectos.

- Ciclo de vida de las tareas.
- Alertas automáticas.
- Auditoría del sistema.

Todos los modelos se inicializan y exportan desde el paquete central de modelos:

app/models/__init__.py

Visión general de relaciones entre entidades

El siguiente diagrama ilustra las relaciones entre los cinco modelos, destacando las restricciones de claves foráneas (Foreign Keys) y el mapeo dentro del espacio de entidades del código.

Diagrama Entidad-Relación (ER) de TaskFlow

(Imagen 1)

Categorías principales de modelos

Modelos User y Project

User

El modelo **User** administra la autenticación y autorización de usuarios, soportando roles como:

- `admin`
- `member`

app/models/user.py

Project

El modelo **Project** funciona como contenedor para el trabajo colaborativo.

Cada proyecto debe tener obligatoriamente un campo:

owner_id

que referencia a un usuario (**User**).

app/models/project.py

Características clave

Reglas de cascada (Cascade Rules)

Las relaciones están configuradas para eliminar automáticamente registros relacionados:

- Al eliminar un **User**, también se eliminan sus proyectos.
- Al eliminar un **Project**, se eliminan todas sus tareas asociadas.

Esto se logra mediante:

`cascade='all, delete-orphan'`

Referencias:

app/models/user.py
app/models/project.py

Serialización

Tanto **User** como **Project** implementan un método:

`to_dict()`

para convertir objetos en diccionarios que puedan enviarse como respuestas JSON de la API.

El modelo **Project** incluye además un parámetro opcional:

`include_tasks`

que permite incluir las tareas asociadas durante la serialización.

Referencia:

app/models/project.py

Para más detalles consultar:

Modelos Task, Notification y LogEntry

Task

El modelo **Task** representa la unidad principal de trabajo dentro del sistema.

Permite gestionar aspectos como:

- Estado (**status**)
- Prioridad (**priority**)
- Archivos adjuntos

Ejemplos de estados:

- **pendiente**
- **completada**

Referencias:

app/models/task.py

Soporte para archivos

Las tareas pueden almacenar documentos asociados mediante el campo:

document_filename
app/models/task.py

Notification

El modelo **Notification** almacena alertas generadas para los usuarios.

Generalmente son creadas automáticamente por servicios en segundo plano para informar eventos como:

- Tareas vencidas.
- Recordatorios.

- Eventos importantes del sistema.

Referencia:

app/models/notification.py

LogEntry

El modelo **LogEntry** implementa la auditoría del sistema.

Registra acciones realizadas utilizando constantes estandarizadas, por ejemplo:

ACTION_TASK_CREATED
ACTION_USER_LOGIN

Referencia:

app/models/log_entry.py

Funciones de soporte de estos modelos

Alertas automáticas

Las entradas de **Notification** son utilizadas para informar a los usuarios sobre eventos relevantes, especialmente aquellos detectados por servicios automáticos.

Ejemplos:

- Tareas vencidas.
- Fechas límite próximas.
- Cambios importantes en proyectos.

Referencia:

app/models/notification.py

Registro de auditoría

Las entradas de **LogEntry** mantienen un historial global de actividades dentro del sistema.

Esto permite:

- Rastrear acciones de usuarios.
- Registrar eventos del sistema.
- Facilitar auditorías y depuración.

Referencia:

`app/models/log_entry.py`

Integración del sistema: Mapeo entre código y entidades

El siguiente diagrama conecta los modelos conceptuales de datos con su implementación dentro de las capas de repositorios y servicios.

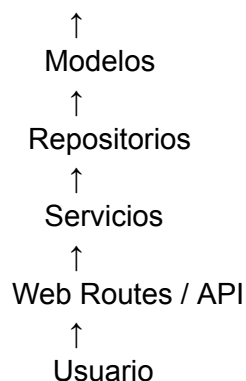
Flujo de integración de modelos (Model Integration Flow)

Este flujo muestra cómo:

1. Los modelos SQLAlchemy representan los datos.
2. Los repositorios encapsulan el acceso a dichos modelos.
3. Los servicios utilizan los repositorios para ejecutar lógica de negocio.
4. Las capas Web y API consumen los servicios para exponer funcionalidad a los usuarios.

En resumen:

Base de Datos SQLite



Esta arquitectura mantiene una clara separación de responsabilidades y facilita el mantenimiento, las pruebas y la reutilización del código.

ahora sigue con: Database and Migrations Relevant source files TaskFlow utilizes a relational database to persist application state, managed through the SQLAlchemy ORM. The system is configured to use SQLite by default, with schema versioning handled by Flask-Migrate (an Alembic wrapper). Database Overview The database file, typically named taskflow.db, is stored within the application's instance folder. The connection and session management are handled by the db object initialized in the models package app/models/__init__.py¹¹

Data Persistence Components	Component	Description	Code Entity Engine
Manages the connection to taskflow.db.	SQLAlchemy Models	Python classes representing database tables.	User, Project, Task, etc.
Migrations	Version control for the database schema.	Flask-Migrate / Alembic	Metadata
	Collection of table objects and their schemas.	db.metadata	

Base de Datos y Migraciones (Database and Migrations)

Archivos fuente relevantes

TaskFlow utiliza una **base de datos relacional** para persistir el estado de la aplicación, gestionada mediante el ORM **SQLAlchemy**.

El sistema está configurado para utilizar **SQLite** de forma predeterminada, mientras que el versionado y evolución del esquema se gestionan mediante **Flask-Migrate**, una capa de integración basada en **Alembic**.

Visión general de la base de datos

El archivo de base de datos, normalmente denominado:

taskflow.db

se almacena dentro de la carpeta **instance** de la aplicación.

La conexión a la base de datos y la gestión de sesiones son manejadas por el objeto:

db

inicializado dentro del paquete de modelos.

Referencia:

Componentes de persistencia de datos

La capa de persistencia está compuesta por varios elementos que trabajan conjuntamente para almacenar y recuperar información.

Componente	Descripción	Entidad de código
Motor (Engine)	Gestiona la conexión con <code>taskflow.db</code> .	SQLAlchemy
Modelos (Models)	Clases Python que representan tablas de la base de datos.	User, Project, Task, etc.
Migraciones (Migrations)	Control de versiones del esquema de la base de datos.	Flask-Migrate / Alembic
Metadatos (Metadata)	Colección de tablas y definiciones de esquema.	<code>db.metadata</code>

Motor de Base de Datos (Engine)

El motor de SQLAlchemy es responsable de:

- Abrir conexiones a la base de datos SQLite.
- Ejecutar consultas SQL.
- Gestionar transacciones.
- Coordinar el acceso a los datos entre la aplicación y SQLite.

Gracias al ORM, los desarrolladores interactúan principalmente con objetos Python en lugar de escribir SQL directamente.

Modelos

Los modelos representan las tablas de la base de datos.

Por ejemplo:

- `User` → usuarios.
- `Project` → proyectos.
- `Task` → tareas.
- `Notification` → notificaciones.
- `LogEntry` → registros de auditoría.

Cada instancia de un modelo corresponde a una fila dentro de la tabla asociada.

Migraciones

Las migraciones permiten modificar la estructura de la base de datos sin perder los datos existentes.

TaskFlow utiliza:

Flask-Migrate

↓

Alembic

para:

- Crear nuevas tablas.
- Agregar columnas.
- Eliminar columnas.
- Modificar restricciones.
- Mantener sincronizados el código y la base de datos.

Esto proporciona un historial de cambios del esquema similar a un sistema de control de versiones para el código fuente.

Metadatos

SQLAlchemy mantiene internamente un conjunto de metadatos accesible mediante:

`db.metadata`

Este objeto contiene información sobre:

- Tablas registradas.
- Columnas.
- Tipos de datos.
- Restricciones.
- Relaciones entre entidades.

Los metadatos son utilizados por SQLAlchemy y Alembic para generar consultas y migraciones automáticamente.

Resumen

La persistencia de datos en TaskFlow se basa en cuatro pilares:

SQLite

↑

SQLAlchemy Engine

↑

Modelos ORM

↑

Flask-Migrate / Alembic

Esta combinación permite que la aplicación almacene información de manera consistente, facilite la evolución del esquema de la base de datos y mantenga una capa de acceso a datos desacoplada del resto del sistema.